

Matrices Final Project

Rohan Shekar, Shyamak Singh

21-241 Fall 2024

1 Introduction

1.1 Abstract

This project aims to investigate the mathematical foundations underscoring the *PageRank* and *HITS* algorithms, two foundational algorithmic models in modern search engine ranking. By applying Linear Algebra techniques, the paper explores the mathematics behind implementation of PageRank (Stochastic (Markov) matrices, eigenvalue decomposition) and analyze its ability to rank web pages based on link structure and user behavior. Subsequently, the paper examines the HITS algorithm, which evaluates pages based on authority and hub scores.

By applying these algorithms on a simplified 5-node network and subsequently more dense networks, the paper analyzes and evaluates the accuracy, ranking methodologies, and applications of each algorithm in web search. This research paper thus highlights the strengths and limitations of each algorithm, offering insights into their roles in the evolution of search engine technology.

2 Rationale

2.1 The Growing Pains of Scaling the Internet

What began in the 1960s as a project by DARPA (Defense Advanced Research Projects Agency) enabling government researchers to share information eventually has since given rise to the internet. Officially born in 1983, this digital infrastructure broke barriers and interconnected the world in ways never seen before. In 1991, the first website ever was launched by Tim Berners-Lee at CERN, marking the beginning of the World Wide Web as we know it today. Growth was slow at first, as in the following year just 10 new websites were added to the internet. However, by 1993, the internet was slowly but surely becoming a vital part of daily life for people worldwide, and consequently the number of websites had surged to 2,278. This exponential growth has yet to stop, and as of today there are over *1.1 billion* websites online, with approximately 252,000 new sites being added every single day.

However, in the early days of the internet whilst it was slowly becoming the world's repository of information, the challenge of accessing it effectively also grew. The immense size of the web rendered simple navigation impractical, and users often struggled to locate the information they needed amidst this rapidly expanding digital platform. By the early 1990s, it became increasingly clear to computer scientists that there needed to be a way to rank websites not only for their relevance but also for their usefulness. Without an efficient way to sift through the ocean of online content, the incredible potential of the internet would remain largely untapped.

2.2 The first steps towards a solution: PageRank

Recognizing this growing problem, Larry Page and Sergey Brin, PhD students at Stanford University, proposed a novel solution. They theorized that the structure of *hyperlinks* between web pages could serve as a reliable measure of a page's importance. Their insight was simple yet revolutionary: links from one page to another could be seen as 'votes of confidence', signaling the linked page's relevance. Furthermore, they posited that all links shouldn't be treated equally—outgoing links from 'important' pages should carry more weight than those from a less significant ones. These two ideas form the foundation of the PageRank algorithm, an implementation that effectively analyzes both the quantity and quality of links when ranking web pages.

PageRank represented a paradigm shift in organization and retrieval of information on the web. By focusing on the inter-connectivity of pages rather than simply their content, PageRank enabled search engines to deliver more accurate and trustworthy results. This implementation simultaneously addressed the immediate challenges of navigating the web but also allowed for scalability as more and more pages get added to the web, setting the stage for the modern internet.

3 PageRank

3.1 Google's First Solution to Ranking Webpages

The PageRank algorithm evaluates and ranks web pages by analyzing the structure of links between them, assigning a measure of importance to each page. The fundamental principle of the algorithm is that a web page is deemed important if it is linked to by other significant pages. Unlike treating all links equally, PageRank assigns greater weight to links coming from pages that themselves have high ranks, creating a recursive method of evaluating importance.

Mathematically, PageRank is modeled as a probability distribution that represents the likelihood of a random web surfer arriving at a given page. In this system, a user begins at a random page and navigates the web by following hyperlinks. Pages that are linked to by multiple sources have a higher probability of being visited, and thus are given more importance in a network.

Let's represent this problem visually using a simple example. Suppose we want have a simplified network of 5 webpages which we need to rank according to importance. We can implement the following network as a graph with the nodes as follows:

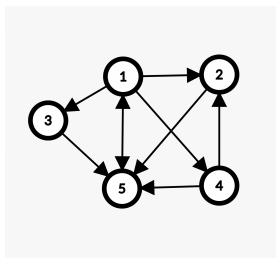


Figure 1: Graph

The graph consists of 5 nodes (P_1, P_2, P_3, P_4, P_5) with the following directed links:

- P_1 links to P_2, P_4, P_5 .
- P_2 links to P_1, P_5 .
- P_3 links to P_1, P_5 .
- P_4 links to P_5 .
- P_5 links to P_3, P_4 .

We have here a vast quantity of information to encode and then use to determine rank. Doing it manually would take longer, and is certainly not viable over the vast scale of the internet. And thus, the rationale for PageRank is born.

3.2 Improving Native Random Walk to reach PageRank

The first steps toward PageRank involve building on the process known as Native Random Walk, defined generally as a stochastic process creating a ‘path’ of successive random steps through some mathematical space. The PageRank algorithm implemented by Larry Page and Sergey Brin builds on Native Random Walk method by addressing some critical assumptions that the latter makes. Native Random Walk assumes that all web pages are interconnected, allowing a user to navigate between any two pages, which does not align with reality as there are many web pages have limited or no outbound links. Secondly, in a naive random walk, all links are treated equally, meaning the importance of a page is determined solely by the number of links it receives.

To address these challenges and ensure the algorithm converges reliably, PageRank implements two key changes, the first of which being the incorporation of a “damping factor.” The damping factor, typically set at 0.85, represents the probability that a user will stop following links and instead jump to a random page. This adjustment handles “dangling nodes” (pages without outbound links) and ensures that every page, even those with no incoming links, has a non-zero rank. By combining the natural navigation of links with the possibility of random jumps, the PageRank algorithm guarantees convergence and provides a realistic representation of how users interact with the web. Finally, PageRank assigns weights to links based on the importance of the linking page, addressing the second issue of native random walk.

3.3 Using a Stochastic Matrices to Implement PageRank

We PageRank algebraically using simple Stochastic (Markov) Matrices. Before we revisit our sample network, let’s define some key terms in Linear Algebra that will be used to implement PageRank:

Eigenvalues: For a given $n \times n$ matrix A and some vector $v \in R^n$, an eigenvalue λ is a scalar such that $Av = \lambda v$.

Eigenvectors: For a given $n \times n$ matrix A and some eigenvalue λ for said matrix, an eigenvector $v \in R^n$ is a vector such that $Av = \lambda v$.

Probability Vectors: A probability vector $v \in R^n$ is a vector whose entries are nonnegative and sum to 1.

Markov (Stochastic) Matrices: Markov Matrices are square matrices whose columns are probability vectors.

Steady-State Vectors: For some Markov Matrix A , a steady-state vector q is a probability vector such that $Aq = q$.

Now that we have defined key terms in Linear Algebra needed to represent PageRank mathematically, we can break down our problem mathematically. Revisiting our example:

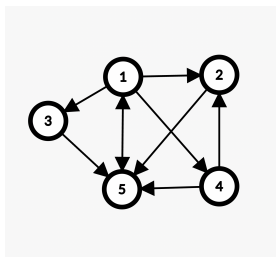


Figure 2: Graph

The graph consists of 5 nodes (P_1, P_2, P_3, P_4, P_5) with the following directed links:

- P_1 links to P_2, P_4, P_5 .
- P_2 links to P_1, P_5 .
- P_3 links to P_1, P_5 .
- P_4 links to P_5 .
- P_5 links to P_3, P_4 .

In order to solve our problem, we can use a matrix to encode the information given above. If we are able to encode the information in such a way that it is stored in a Markov Matrix, the corresponding steady-state vector would give us an estimate of the probability that a random user ends up on either of the webpages, thus allowing us to rank the webpages effectively.

Let M be a 5×5 matrix representing the connectivity of the different nodes in our example, where the rows and columns of the matrix represent the different web pages. Note that this matrix is not a symmetric matrices since connections between nodes on our graph are not bi-directional. We can use simple 0s and 1s to represent the information: in our matrix; the $(i, j)^{th}$ entry corresponds to whether or not node j ‘points’ (ie, has a link) to node i , with 1–entries indicating a connection and 0–entries indicating no connectivity between the pages. We therefore have it that

$$M = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

However, our primary aim is to encode the information in a stochastic matrix. By definition, the entries in each columns of stochastic matrices sum to 1.

Therefore, we must *normalize* M , leading us to the following representation:

$$M_n = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0.25 & 0 & 0 & 0.5 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 1 & 1 & 0.5 & 0 \end{bmatrix}$$

Note that we have to modify the normalization function to account for dangling nodes in order to ensure that the columns remain probability vectors, since the row and column corresponding to the dangling node will be full of 0s. Refer to the second example on page 10 to see how this is implemented.

We have thus have our desired Markov Matrix. In order to ensure convergence to a unique steady state vector (i.e. only one possible solution for probability of a random surfer ending up on each site), we must prove that it is a positive regular Markov matrix. Consider M_n^2 :

$$M_n^2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0.25 & 0 & 0 & 0.5 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 1 & 1 & 0.5 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0.25 & 0 & 0 & 0.5 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 1 & 1 & 0.5 & 0 \end{bmatrix} = \begin{bmatrix} 0.25 & 1 & 1 & 0.5 & 0 \\ 0.125 & 0.25 & 0.25 & 0.5 & 0 \\ 0.125 & 0 & 0 & 0 & 0 \\ 0.125 & 0 & 0 & 0 & 0 \\ 0.375 & 1 & 1 & 0.75 & 0 \end{bmatrix}$$

M_n^2 is positive, therefore it follows that $\forall k > 0$, M_n^k is positive, and therefore M_n is a positive Markov Matrix.

In order to proceed further from here, we must implement the modifications introduced by PageRank on the Native Random Walk implementation.

Let's implement the damping factor. Suppose $d = 0.85$. This implies that 85% of the time, the users follow links when traversing the network, and $100 - 85 = 15\%$ of the time, they jump to a random node. Given that this jump can happen to any of the nodes, we define the matrix describing jumps as a normalized Stochastic Matrix, whose entries will therefore be $1 \div \{\text{number of nodes}\}$. Let's call this matrix $M_{1/n}$.

Therefore, incorporating the dampening factor into our Markov Matrix M_n yields the following expression:

$$M_d = d \cdot M_n + (1 - d)M_{1/n}$$

$$\text{where } M_{1/n} = \begin{bmatrix} 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \end{bmatrix}$$

Thus, our expression for M_d is:

$$M_d = (0.85) \cdot \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0.25 & 0 & 0 & 0.5 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 0 & 0 & 0 & 0 \\ 0.25 & 1 & 1 & 0.5 & 0 \end{bmatrix} + (0.15) \cdot \begin{bmatrix} 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \end{bmatrix}$$

It can clearly be seen that even for simple situations like a 5-node network, the calculations can become tedious to perform by hand. Thus, we use the programming language *Julia* to execute our calculations. Julia is a programming language well-suited for linear algebra and numerical computations, with built-in support for matrix operations, efficient handling of sparse and dense arrays, and integration of various libraries.

3.4 Implementing PageRank with Julia

To model our situation using Julia, we use the following code block:

```
using LinearAlgebra

# Define the normalized stochastic matrix M_n
M_n = [
    0 0 0 0 1;
    0.25 0 0 0.5 0;
    0.25 0 0 0 0;
    0.25 0 0 0 0;
    0.25 1 1 0.5 0
]

# Define the damping factor and the uniform probability matrix
d = 0.85
n = size(M_n, 1)
uniform_matrix = ones(n, n) / n

# Compute the final matrix M
M = d * M_n + (1 - d) * uniform_matrix
```

The result gives us the following Stochastic Matrix:

$$M_d = \begin{bmatrix} 0.0300 & 0.0300 & 0.0300 & 0.88 & 0.8800 \\ 0.2425 & 0.0300 & 0.0300 & 0.4500 & 0.0300 \\ 0.2425 & 0.0300 & 0.0300 & 0.0300 & 0.0300 \\ 0.2425 & 0.0300 & 0.0300 & 0.0300 & 0.0300 \\ 0.2425 & 0.8800 & 0.08800 & 0.4550 & 0.0300 \end{bmatrix}$$

We can verify that the given matrix is indeed a Stochastic Matrix using the following block of code:

```

# Define the matrix M
M = [
    0.03  0.03  0.03  0.03  0.88;
    0.2425 0.03  0.03  0.455 0.03;
    0.2425 0.03  0.03  0.03  0.03;
    0.2425 0.03  0.03  0.03  0.03;
    0.2425 0.88  0.88  0.455 0.03
]
# Verify column sums
println("\nColumn sums of M: ", sum(M, dims=1))

```

We have the code output to be the following line:

‘Column sums of M: [1.0 1.0 1.0 1.0 1.0]’

Confirming that the entries are non-negative and that the columns sum to 1 indicates that we have successfully implemented the dampening factor. Now, to calculate the steady-state vector, we must calculate the eigenvalues of matrix M_d , and find the eigenvector associated with eigenvalue $\lambda = 1$. The typical procedure from here is to apply the characteristic polynomial, ie

$$\det(A - \lambda I) = 0$$

From here, we solve for λ . Since our matrix is quite large, we once again employ Julia to proceed from here:

```

using LinearAlgebra

# Define the matrix M directly from the image
M = [
    0.03  0.03  0.03  0.03  0.88;
    0.2425 0.03  0.03  0.455 0.03;
    0.2425 0.03  0.03  0.03  0.03;
    0.2425 0.03  0.03  0.03  0.03;
    0.2425 0.88  0.88  0.455 0.03
]

# Calculate eigenvalues and eigenvectors
eig_result = eigen(M) # Store the result to avoid conflict with
# 'eigen' function
eigenvalues = real(eig_result.values)
eigenvectors = real(eig_result.vectors)

# Print the eigenvalues
println("Eigenvalues:")
for (i, value) in enumerate(eigenvalues)
    println("Eigenvalue $i: $value")
end

```



```

# Print the eigenvectors
println("\nEigenvectors:")
for (i, vector) in enumerate(eachcol(eigenvectors))
    println("Eigenvector $i: $vector")
end

```

The code output gives us the following eigenvalues:

$$\lambda = -0.33359, -0.18282, 5.648691133015555e - 17, 1$$

With the eigenvalue $\lambda = -0.33359$ having algebraic multiplicity of 2. Since we are interested in finding the steady-state vector, we are primarily interested in the eigenvector associated with $\lambda = 1$. In our given example and in many other cases, the raw eigenvectors we get from our Stochastic Matrix often contain imaginary components. Thus, in order to extract the real component and the useful information, we can use Julia once again and update our existing code interface to give us our desired vector:

```

using LinearAlgebra

# Define the matrix M directly
M = [
    0.03 0.455 0.88 0.88 0.2425;
    0.03 0.03 0.03 0.03 0.2425;
    0.03 0.03 0.03 0.03 0.2425;
    0.03 0.455 0.03 0.03 0.2425;
    0.88 0.03 0.03 0.03 0.03
]

# Calculate eigenvalues and eigenvectors
eig_result = eigen(M) # Eigen decomposition
eigenvalues = real(eig_result.values) #Real Component of Eigenvalue
eigenvectors = eig_result.vectors

# Identify the eigenvector corresponding to the eigenvalue closest
# to 1
tolerance = 1e-6 # Since eigenvalue may not be exactly =1
index = argmin(abs.(eigenvalues .- 1)) # Find index of eigenvalue
# closest to 1
principal_eigenvector = eigenvectors[:, index] # Get the
# corresponding eigenvector
#We successfully have our steady-state vector

# Normalize the eigenvector to ensure it sums to 1 (PageRank
# requirement)
pagerank_vector = real(principal_eigenvector) # Take the real part
pagerank_vector /= sum(pagerank_vector) # Normalize to sum to 1

# Print results
println("Eigenvalues:")

```

```
println(eigenvalues)

println("\nPrincipal Eigenvector (PageRank Scores):")
println(pagerank_vector)
```

And thus, after all our operations, we have our pagerank vector v to be

$$v = \begin{bmatrix} 0.32109 \\ 0.13998 \\ 0.09823 \\ 0.09823 \\ 0.34246 \end{bmatrix}$$

In conclusion, in our 5-node network, the ranking of nodes in terms of importance is as follows: P_5, P_1, P_3, P_4, P_2 .

3.5 Applying Algorithm to Complex Networks

One can note that the previously-chosen example did not take into account any disconnected nodes. To demonstrate that PageRank works on such networks as well, let's extend our network and consider a more complex example with disconnected nodes.

Consider the following network:

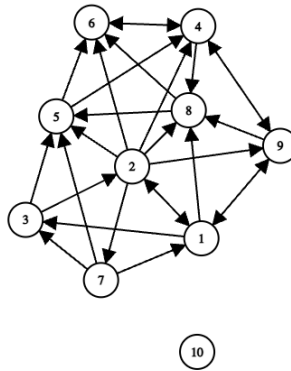


Figure 3: Network 2

Describing our image, we have it that:

- Node P_1 links to P_2, P_3, P_8, P_9
- Node P_2 links to $P_1, P_4, P_5, P_6, P_7, P_8, P_9$

- Node P_3 links to P_0, P_5
- Node P_4 links to P_6, P_8, P_9
- Node P_5 links to P_4, P_6
- Node P_6 links to no nodes
- Node P_7 links to P_1, P_3, P_5
- Node P_8 links to P_5, P_6
- Node P_9 links to P_1, P_4, P_8
- Node P_{10} is **isolated**, with no incoming or outgoing links.

Drawing a matrix would become very tedious and difficult. Thus, we can combine the different segments of Julia code discussed earlier into one code block that takes in these inputs and outputs our desired PageRank vector. The code-block that will be used is as follows:

```
using LinearAlgebra

function create_adjacency_matrix(connections::Vector{Vector{Int}},
    num_nodes::Int)
    # Initialize an n x n matrix with zeros
    adjacency_matrix = zeros{Int}(num_nodes, num_nodes)

    # Fill in the adjacency matrix based on the connections
    print(length(connections))
    for j in 1:length(connections)
        for i in connections[j]
            print(j)
            adjacency_matrix[i, j] = 1
        end
    end
    return adjacency_matrix
end

function normalize_matrix(adjacency_matrix::Matrix{Int})
    # Copy the adjacency matrix to a new matrix for normalization
    normalized_matrix = Float64{adjacency_matrix}
    # Normalize each column to ensure column sums equal 1
    num_nodes = size(adjacency_matrix, 1)
    col_sum = sum(adjacency_matrix, dims=1)
    for j in 1:num_nodes
        if col_sum[j] > 0
            normalized_matrix[:, j] /= col_sum[j]
            #continue
        else
            #Dangling / Isolated Nodes
            normalized_matrix[:,j] .= 1 / (num_nodes - 1)
            normalized_matrix[j,j] = 0
        end
    end
    return normalized_matrix
end
```

```

end

# Define the connections as described in the image
connections = [
    Int[2, 3, 8, 9], # Node P1 links to P2, P3, P8, P9
    Int[1, 4, 5, 6, 7, 8, 9], # Node P2 links to P1, P4, P5, P6, P7,
    P8, P9
    Int[2, 5], # Node P3 links to P2, P5
    Int[6, 8, 9], # Node P4 links to P6, P8, P9
    Int[4, 6], # Node P5 links to P4, P6
    Int[4], # Node P6 links to P4
    Int[1, 3, 5], # Node P7 links to P1, P3, P5
    Int[5, 6], # Node P8 links to P5, P6
    Int[1, 4, 8], # Node P9 links to P1, P4, P8
    Int[] # Node P10 is isolated
]

# Number of nodes
num_nodes = 10

# Create the adjacency matrix
adjacency_matrix = create_adjacency_matrix(connections, num_nodes)
display(adjacency_matrix)

# Normalize the adjacency matrix to create a stochastic matrix
stochastic_matrix = normalize_matrix(adjacency_matrix)
display(stochastic_matrix)

# Define the damping factor and the uniform probability matrix
d = 0.85

uniform_matrix = ones(num_nodes, num_nodes) / num_nodes

# Compute the final matrix M
M = d * stochastic_matrix + (1 - d) * uniform_matrix
display(M)

# Calculate eigenvalues and eigenvectors
eig_result = eigen(M) # Eigen decomposition
eigenvalues = real(eig_result.values) # Take the real part
eigenvectors = real(eig_result.vectors) # Take the real part

# Identify the eigenvector corresponding to the eigenvalue closest
to 1
tolerance = 1e-6
index = argmin(abs.(eigenvalues .- 1)) # Find index of eigenvalue
closest to 1
principal_eigenvector = eigenvectors[:, index] # Get the

```

```

corresponding eigenvector

# Normalize the eigenvector to ensure it sums to 1 (PageRank
  requirement)
pagerank_vector = principal_eigenvector
pagerank_vector /= sum(pagerank_vector) # Normalize to sum to 1

# Print results
println("Eigenvalues:")
println(eigenvalues)
println("\nPrincipal Eigenvector (PageRank Scores):")
println(pagerank_vector)

# Verify that the pagerank_vector is a probability vector
println("\nSum of PageRank vector: ", sum(pagerank_vector)) # Should
  be close to 1
println("All entries non-negative: ", all(pagerank_vector .>= 0))
  #Should be true

```

Note the code in the function *normalize_matrix*. Whenever the condition $col_sum[j] > 0$ fails (i.e. dangling node), we add a condition to ensure the column entries except for the diagonal entry are $\frac{1}{n-1}$, as it has an equal chance to go to this page during a random jump.

And thus, our final desired PageRank vector v is

$$v = \begin{bmatrix} 0.05917669573761808 \\ 0.04393241426319147 \\ 0.035154587652425864 \\ 0.26827726001104135 \\ 0.10303521319399407 \\ 0.20173349624410705 \\ 0.02175131697005422 \\ 0.14160076077680833 \\ 0.11033825515075961 \\ 0.015000000000000003 \end{bmatrix}$$

Which we interpret to rank our pages in the following order of importance:

$$P_4, P_6, P_8, P_9, P_5, P_1, P_2, P_3, P_7, P_{10}$$

(Note that the last 2 lines of code have been included to verify that the PageRank vector is indeed a probability vector)

3.6 PageRank Extension on Random Matrices

In our previous two examples, we used fixed-sized matrices and representations to evaluate PageRank. Note that it is possible to generate an arbitrary $n \times$

n matrix representing a network which we can test PageRank on. The code enabling this is attached below:

```
using LinearAlgebra

function create_adjacency_matrix(connections::Vector{Vector{Int}},
    num_nodes::Int)
    # Initialize an n x n matrix with zeros
    adjacency_matrix = zeros{Int}(num_nodes, num_nodes)

    # Fill in the adjacency matrix based on the connections
    for i in 1:length(connections)
        for j in connections[i]
            adjacency_matrix[i, j] = 1 # Adjust column index from
                0-based to 1-based
        end
    end
    return adjacency_matrix
end

function normalize_matrix(adjacency_matrix::Matrix{Int})
    println("Adjacency Matrix Before Normalization:")
    for row in eachrow(adjacency_matrix)
        println(row)
    end
    # Copy the adjacency matrix to a new matrix for normalization
    normalized_matrix = Float64.(adjacency_matrix)
    # Normalize each column to ensure column sums equal 1
    num_nodes = size(adjacency_matrix, 1)
    for j in 1:num_nodes
        col_sum = sum(adjacency_matrix[:, j])
        if col_sum > 0
            normalized_matrix[:, j] /= col_sum
        else
            # Handle dangling nodes (columns with sum 0)
            normalized_matrix[:, j] .= 1 / num_nodes
        end
    end
    return normalized_matrix
end

# Define the connections as described in the image
connections = [
    Int[], # Node P1 Links to P1, P2, P4, P6, P8
    Int[3, 5], # Node P2 links to P3, P9
    Int[2, 4, 5], # Node P3 links to P3, P6, P7, P8
    Int[1, 5],
    Int[3]
]
```

```

# Number of nodes
num_nodes = 5

# Create the adjacency matrix
adjacency_matrix = create_adjacency_matrix(connections, num_nodes)

# Normalize the adjacency matrix to create a stochastic matrix
stochastic_matrix = normalize_matrix(adjacency_matrix)
display(stochastic_matrix)

# Define the damping factor and the uniform probability matrix
d = 0.85

uniform_matrix = ones(num_nodes, num_nodes) / num_nodes

# Compute the final matrix M
M = d * stochastic_matrix + (1 - d) * uniform_matrix
display(M)

# Calculate eigenvalues and eigenvectors
eig_result = eigen(M) # Eigen decomposition
eigenvalues = eig_result.values
eigenvectors = eig_result.vectors

# Identify the eigenvector corresponding to the eigenvalue closest
# to 1
tolerance = 1e-6
index = argmin(abs.(eigenvalues .- 1)) # Find index of eigenvalue
# closest to 1
principal_eigenvector = eigenvectors[:, index] # Get the
# corresponding eigenvector

# Normalize the eigenvector to ensure it sums to 1 (PageRank
# requirement)
pagerank_vector = real(principal_eigenvector) # Take the real part
pagerank_vector /= sum(pagerank_vector) # Normalize to sum to 1

# Print results
println("Eigenvalues:")
println(eigenvalues)
println("\nPrincipal Eigenvector (PageRank Scores):")
println(pagerank_vector)

# Verify that the pagerank_vector is a probability vector
println("\nSum of PageRank vector: ", sum(pagerank_vector)) # Should
# be close to 1
println("All entries non-negative: ", all(pagerank_vector .>= 0))
# Should be true

```

(Refer to the comments to see the exact line of code to change to specify the number of nodes, as well as the damping factor)

And thus, we have successfully implemented PageRank, as envisioned by Sergey Brin and Larry Page many decades ago. However, this is not the only solution to ranking web pages- this too has its limitations. While this is a start, there have been newer and improved algorithms implemented since then, which we will see in the following sections.

4 HITS

4.1 PageRank’s successor: The HITS Algorithm

When a user gives a query, the most intuitive results to give are pages ranked in order of how much their text matches the query. This approach, however, allows for the search algorithm to miss important results that don’t use words in the query and instead use synonymous words. For example, if a person was to search for “best pieces of literature to buy,” the algorithm could miss a great article called “Best Books of 2024” that only uses the word books and not the word literature. To avoid this, Jon Kleinberg created the HITS algorithm.

The HITS algorithm relies on the internet’s systems of hyperlinks, specifically the concepts of inlinks and outlinks. The algorithm uses these to optimally find two important types of web pages based on a query: authorities and hubs. Authorities are the main websites that give the desired information for a specific query. Hubs can be thought of as pages that help in getting the authority pages and have links to these authority pages. Our goal is to efficiently get quality authorities and hubs.

4.2 Implementing HITS

The algorithm begins with taking the first 200 document that have the highest frequencies of the words in the query. Even though this is what the algorithm aims to avoid, the results give a reasonable base. It is important to note here that these pages will not be highly connected so using PageRank would be ineffective. The algorithm then forms a sub-graph of the internet taking all pages that point to or from these initial pages. This becomes the base graph for HITS.

Recall the example we used to describe PageRank. Let's suppose that our result was modeled by the following nodes:

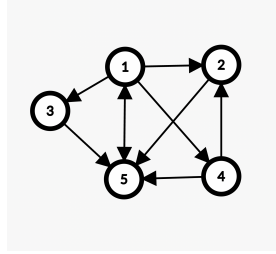


Figure 4: Graph

The first step in the algorithm is writing the graph in an n by n adjacency matrix L where n is the number of nodes. There is a zero in row i column j if i does not point to j and if and there is a 1 in row i column j if i points to j :

$$L = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Next, we create our authority weight vector and hub weight vectors, call them a and h accordingly. The i th entry in the authority vector corresponds to the authority value of the i th node. The i th entry in the hub weight vector corresponds to the hub value of the i th node.

We can initialize a and h to any positive vector, all ones is typically used:

$$a = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$h = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

To apply the algorithm, we follow the following definitions. The authority value of a node is the sum of all the hub values of all the nodes pointing to the node. The hub value of a node is the sum of all the authority values of nodes that a node points to.

Written in mathematical notation:

Take E to be the set of all nodes that node i points to and D to be the set of all nodes that points to node i .

$$a_i^k = \sum_{\forall j \in E} h_j^{k-1}$$

$$h_i^k = \sum_{\forall j \in D} a_j^k$$

Where a^k and h^k are the k th iteration of a and h respectively. From this, we get the process of the HITS algorithm:

1. Calculate the authority values of each node based on the hub values of nodes pointing to each node.
2. Calculate the hub values of each node based on the authority values of the nodes it points to.
3. Normalize both vectors so that the sum of their entries add to 1.

Here is an example of a few iterations of this process:

	Init Auth	Init Hub	Auth 1	Hub 1	Auth 2	Hub 2
Node 2	1	1	1/9	8/23	1/53	52/133
Node 2	1	1	2/9	4/23	14/53	22/133
Node 2	1	1	1/9	4/23	8/53	22/133
Node 2	1	1	1/9	6/23	8/53	36/133
Node 2	1	1	4/9	1/23	22/53	1/133

The algorithm's conclusion comes from the fact that this process typically converges so we can check for a when the authority and hub vectors converge to a steady value. One way to do this is to see if the norm of the difference between the past vectors and the current vectors is smaller than a defined value.

Now let's apply the code for this algorithm:

Here's how we would structure the algorithm in Julia:

1. Initialize the adjacency matrix, the authority weight vector and the hub weight vector.
2. Create variables that hold the past iterations of a and h (a_0 and h_0) and initialize them to all zeros (these values do not matter as long as they are different from the initial a and h)

3. Enter a while loop which should check if the different of the past iteration vector and current iteration vectors has a norm less than 0.001. (The smaller the number, the more accurate the vectors but the longer the algorithm will take)
4. Update the past iteration vectors to the current ones.
5. Update all adjacency values according to hub values. Then, update all hub values according to adjacency values.
6. Normalize a and h by dividing each entry by the sum of all entries.
7. Repeat the while loop.

4.3 Encoding HITS with Julia

Similar to modeling PageRank, we once more turn to Julia to implement the HITS function. Encoding the previously-described information into code format gives us the following code that can be applied to calculate the rank of each webpage:

```
#Import the LinearAlgebra library
using LinearAlgebra

function create_adjacency_matrix(connections::Vector{Vector{Int}},
    num_nodes::Int)
    # Initialize an n x n matrix with zeros
    adjacency_matrix = zeros{Int}(num_nodes, num_nodes)
    # Fill in the adjacency matrix based on the connections
    for i in 1:length(connections)
        for j in connections[i]
            adjacency_matrix[i, j] = 1
        end
    end
    return adjacency_matrix
end

#This function normalizes the passed in vector by deviding every entry
#by the sum of all entries
function normal(a)
    asum = 0.0
    for i in 1:size(a, 1)
        asum = asum + a[i];
    end
    for i in 1:size(a, 1)
        a[i] = a[i] / asum;
    end
end

#L = [0 1 1 1 1; 0 0 0 0 1; 0 0 0 0 1; 0 1 0 0 1 ; 1 0 0 0 0]
```

```

connections = [
    Int[2, 3, 4, 5], # Node P1 Links to P2, P3, P4, P5
    Int[5], # Node P2 links to P5
    Int[5], # Node P3 links to P5
    Int[2, 5], # Node P4 links to P2, P5
    Int[1], # Node P5 links to P1
]

# Number of nodes
num_nodes = length(connections)

# Create the adjacency matrix
adjacency_matrix = create_adjacency_matrix(connections, num_nodes)

#a and h represent the current authoritative and hub vecotrs respectively
a = ones(num_nodes)
h = ones(num_nodes)

#ao and ho represent the previous iterations of the authoritative and hub
vectors.
#They can be initialized to any values as long as they are different
from a and h
ao = 0*ones(num_nodes)
ho = 0*ones(num_nodes)

#Loop while the normals of the differnece between old and new matrices
are greater than 0.01
while(norm(a-ao) > 0.001 && norm(h-ho) > 0.001)
    # Update the previous authoritative and hub wieght vectors to be the
    current vectors
    ao = copy(a)
    ho = copy(h)

    #Update every authoratative weight entry
    for i in 1:size(a, 1)
        a[i] = 0;
        for j in 1:size(a, 1)
            if (adjacency_matrix[j, i] == 1)
                a[i] = a[i] + h[j]
            end
        end
    end

    #Update every hub weight entry
    for i in 1:size(a, 1)
        h[i] = 0;
        for j in 1:size(a, 1)
            if (adjacency_matrix[i, j] == 1)
                h[i] = h[i] + a[j]
            end
        end
    end
end

```

```

        end
    end

    #Normalize the vectors
    normal(a)
    normal(h)
end

#Print the output
println("The final authority weight vector:")
println(a)
println("Authoritative Hub Sorted")
println(sortperm(a, rev=true))
println()
println("The final hub weight vector:")
println(h)
println("Authoritative Hub Sorted")
println(sortperm(h, rev=true))

```

This code gives us the following results for our situation:

$$a = \begin{bmatrix} 8.116224332440547 \cdot 10^{-5} \\ 0.2720558396234072 \\ 0.16232448664881094 \\ 0.16232448664881094 \\ 0.40321402483564645 \end{bmatrix}$$

$$h = \begin{bmatrix} 0.4029172253654708 \\ 0.16247506295581646 \\ 0.16247506295581646 \\ 0.2720999444026556 \\ 3.270432024070379 \cdot 10^{-5} \end{bmatrix}$$

From this we can now state the authoritative and hub rankings of our nodes:

Authoritative: $P_5, P_2, (P_3, P_4), P_1$

Hub: $P_1, P_4, (P_2, P_3), P_5$

Generally looking at our graph, we see that this makes sense, 5 has the most nodes pointing to it, 2 has the second most and 1 is pointing to the most nodes. Node 2 has fairly high both hub and authoritative scores so it would be a good site to recommend.

Similar to PageRank, it is important to not that the previous example did not take into account disconnected nodes. To observe how the HITS algorithm approaches such networks, we utilize the following sample network:

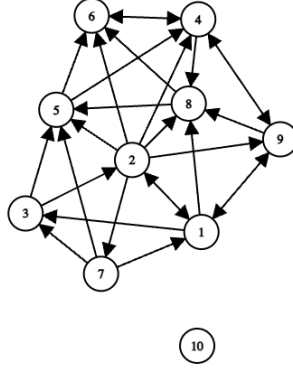


Figure 5: Network 2

This network is represented using the following structure which can be turned into a matrix as done previously:

- Node P_1 links to P_2, P_3, P_8, P_9
- Node P_2 links to $P_1, P_4, P_5, P_6, P_7, P_8, P_9$
- Node P_3 links to P_2, P_5
- Node P_4 links to P_6, P_8, P_9
- Node P_5 links to P_4, P_6
- Node P_6 links to P_4
- Node P_7 links to P_1, P_3, P_5
- Node P_8 links to P_5, P_6
- Node P_9 links to P_1, P_8, P_4
- Node P_{10} is **isolated**, with no incoming or outgoing links.

Using a similar process, we can calculate the adjacency and hub vectors to be the following:

$$a = \begin{bmatrix} 0.12548256707929029 \\ 0.043779932312362796 \\ 0.05421850768884733 \\ 0.13356009875607852 \\ 0.12693733829481346 \\ 0.14588882443699963 \\ 0.06922790582658232 \\ 0.16678439001548162 \\ 0.13412043558954406 \\ 0.0 \end{bmatrix}$$

$$h = \begin{bmatrix} 0.11954964636020249 \\ 0.2703261086377043 \\ 0.05116325457413822 \\ 0.13390219500292105 \\ 0.08374967773878668 \\ 0.04002740501473926 \\ 0.09189825454654398 \\ 0.08176486402744917 \\ 0.12761859409751486 \\ 0.0 \end{bmatrix}$$

From this we can now state the authoritative and hub rankings of our original nodes:

Authoritative: $P_8, P_6, P_9, P_4, P_5, P_1, P_7, P_3, P_2, P_{10}$

Hub: $P_2, P_4, P_9, P_1, P_7, P_5, P_8, P_3, P_6, P_{10}$

Looking at our graph, these results make sense, 8 is one of the most pointed to nodes and it is pointed by other highly pointed to nodes. Additionally, 2 is pointing to the most nodes and has the highest hub ranking. We take node 9 to be one of the best websites according to this algorithm as it is both third in authoritative and hub rankings.

4.4 Further Exploration of the HITS Algorithm

Following these results, it is reasonable to question why this algorithm works and why the authoritative and hub vector specifically converge.

We can rewrite our summation expressions using matrix multiplication:

$$\begin{aligned} a^k &= L^T h^{(k-1)} \\ h^k &= L a^k \\ \Rightarrow a^k &= L^T L a^{(k-1)} \\ \Rightarrow h^k &= L L^T h^{k-1} \end{aligned}$$

These two equations resemble the power method which we can now take advantage of

First, let's clarify some facts:

- LL^T and $L^T L$ only have real eigenvalues as discussed in lecture
- Perron Frobenius tells us that if A is a primitive then:
 - The largest eigenvalue of A, λ_1 , is positive with multiplicity 1
 - Other eigenvalues are less than λ_1

– λ_1 has an eigenvector with all positive entries

[13](Tanase, Remus, 2009)

So, knowing that our matrix is primitive gives us that we have a largest dominating eigenvalue. Perron Frobenius accordingly tells us that this largest eigenvector will dominate and that the series of $z \rightarrow Az \rightarrow A^2z \rightarrow A^kz$ normalized at every step will converge to the eigenvector corresponding to the largest eigenvalue of A normalized. This is known as the Power Algorithm.

To understand how this works, since in our situation, A is symmetric, we can write z as a linear combination of the eigenvectors of A :

$$z = c_1v_1 + c_2v_2 + c_3v_3\ldots + c_nv_n$$

Then, $A^kz =$

$$A^kz = c_1A^kv_1 + c_2A^kv_2 + c_3A^kv_3\ldots + c_nA^kv_n$$

Meaning,

$$A^kz = c_1\lambda_1^kv_1 + c_2\lambda_2^kv_2 + c_3\lambda_3^kv_3\ldots + c_n\lambda_n^kv_n$$

Assuming λ_1 is the largest eigenvalue, which must exist in our scenario, $c_1\lambda_1^kv_1$ dominates and normalizing gives us that A^kz goes to the normalized eigenvector of the largest eigenvalue of A .

Going back to the HITS algorithm, this means that as $k \rightarrow \infty$:

$$\Rightarrow a^k = v_1$$

$$\Rightarrow h^k = v_2$$

Where v_1 is the eigenvector of the largest eigenvalue of L^TL and v_2 is the eigenvector of the largest eigenvalue of LL^T . This is assuming that L^TL and LL^T are primitive. This is because the HITS algorithm does not specifically consider whether or not LL^T , relying on the structure of the web.

Thus, we can structure code that gets the largest eigenvalues of L^TL and LL^T and normalizes them to get the authoritative and hub weight vectors accordingly:

```
#Import the LinearAlgebra library
using LinearAlgebra

function create_adjacency_matrix(connections::Vector{Vector{Int}},
    num_nodes::Int)
    # Initialize an n x n matrix with zeros
```

```

adjacency_matrix = zeros(Int, num_nodes, num_nodes)
# Fill in the adjacency matrix based on the connections
for i in 1:length(connections)
    for j in connections[i]
        adjacency_matrix[i, j] = 1
    end
end
return adjacency_matrix
end

#This function normalizes the passed in vector by deviding every entry
#by the sum of all entries
function normal(a)
    asum = 0.0
    for i in 1:size(a, 1)
        asum = asum + a[i];
    end
    for i in 1:size(a, 1)
        a[i] = a[i] / asum;
    end
end

#L = [0 1 1 1 1; 0 0 0 0 1; 0 0 0 0 1; 0 1 0 0 1 ; 1 0 0 0 0]
connections = [
    Int[2, 3, 4, 5], # Node P1 Links to P2, P3, P4, P5
    Int[5], # Node P2 links to P5
    Int[5], # Node P3 links to P5
    Int[2, 5], # Node P4 links to P2, P5
    Int[1], # Node P5 links to P1
]

# Number of nodes
num_nodes = length(connections)

# Create the adjacency matrix
adjacency_matrix = create_adjacency_matrix(connections, num_nodes)

#Get the eigenvector associated to the largest eigenvalue of (L^T)L
a = eigen(((adjacency_matrix')*(adjacency_matrix))).vectors[:,
    size(adjacency_matrix, 1)]

#Get the eigenvector associated to the largest eigenvalue of L(L^T)
h = eigen(((adjacency_matrix)*(adjacency_matrix'))).vectors[:,
    size(adjacency_matrix, 1)]

#Normalize a and h
normal(a)
normal(h)

#Print the output

```

```
println("The final authority weight vector:")
println(a)
println("Authoritative Hub Sorted")
println(sortperm(a, rev=true))
println()
println("The final hub weight vector:")
println(h)
println("Authoritative Hub Sorted")
println(sortperm(h, rev=true))
```

This code gives us the following results for figure 1:

$$a = \begin{bmatrix} 0.0 \\ 0.2720991538039613 \\ 0.16243456471667694 \\ 0.162434564716677 \\ 0.4030317167626847 \end{bmatrix}$$

$$h = \begin{bmatrix} 0.4030317167626847 \\ 0.16243456471667703 \\ 0.16243456471667705 \\ 0.2720991538039612 \\ 0.0 \end{bmatrix}$$

Following, the authoritative and hub rankings of our nodes are:

Authoritative: 5, 2, (4, 3), 1

Hub: 1, 4, (3, 2), 5

Additionally for figure 2, we get:

$$a = \begin{bmatrix} 0.12546221142913 \\ 0.04370294263232578 \\ 0.05413361472662909 \\ 0.1336683685711473 \\ 0.12678415870677 \\ 0.14595470525557305 \\ 0.06924697454104635 \\ 0.16687279160822313 \\ 0.13417423252915514 \\ 0.0 \end{bmatrix}$$

$$h = \begin{bmatrix} 0.11953539583996607 \\ 0.27035573593653967 \\ 0.051090704379780555 \\ 0.13395519681129575 \\ 0.0837960155955237 \\ 0.04005705446309887 \\ 0.09181439013007334 \\ 0.08173299071970212 \\ 0.1276625161240199 \\ 0.0 \end{bmatrix}$$

From this we can now state the authoritative and hub rankings of our nodes:

Authoritative: $P_8, P_6, P_9, P_4, P_5, P_1, P_7, P_3, P_2, P_{10}$

Hub: $P_2, P_4, P_9, P_1, P_7, P_5, P_8, P_3, P_6, P_{10}$

Comparing the eigenvalue to the iterative methods, we see that the vectors are almost exactly the same, with differences attributed to rounding. Additionally, we see that the ranking are the same from both methods. From this we can see our results support the theory of the HITS algorithm and we see a different way to approach the HITS algorithm.

5 Comparative Analysis: PageRank versus HITS

5.1 Comparing Experiment Results

From the 10-node network example, the two algorithms outputted the following page rankings (from most to least important):

PageRank: $P_4, P_6, P_8, P_9, P_5, P_1, P_2, P_3, P_7, P_{10}$

HITS (Authoritative): $P_8, P_6, P_9, P_4, P_5, P_1, P_7, P_3, P_2, P_{10}$

HITS (Hub): $P_2, P_4, P_9, P_1, P_7, P_5, P_8, P_3, P_6, P_{10}$

As can be seen from the outputs, the two algorithms yielded different rankings. Both algorithms identified the dangling node as the least significant node, yet the outputs differed from there. PageRank and Authoritative-centric HITS algorithm were very similar in ranking pages, sharing commonalities in terms of their top 5 ranked pages, but had significantly different results when compared to the Hub-centric HITS algorithm. Nodes like P_4 were ranked consistently across algorithms, but nodes like P_6 were ranked as high as 2nd on PageRank but 9th on HITS Hub-centric. This distinction highlights the robustness of HITS, as PageRank's limitations to hyper-linkage failed to capture the fact node 6 has no outgoing nodes, meaning P_6 a strong source but does not necessarily have strong links to other nodes. For this reason, it would be preferable to use HITS in certain circumstances.

5.2 Abstract

Both PageRank and HITS are widely recognized and effective algorithms that have been successfully applied (in various forms) in search engines such like Google, Safari, and more. However, it is worth noting that each algorithm has its pros and cons when compared to the other.

PageRank provides a robust method of ranking web pages by leveraging the interconnected structure of hyperlinks to assess importance. A major advantage is its ability to assign more weight to links from authoritative pages, creating a recursive feedback loop that emphasizes *quality* over *quantity*. By introducing a damping factor, it ensures convergence even in networks with isolated nodes, making it applicable over diverse web structures. However, PageRank has limitations. Its reliance on link structure can lead to manipulation through link farming or artificially inflating rankings. Furthermore, it does not consider the actual contents of the page itself, arguably the more important component of a webpage compared to hyperlinks. The algorithm's computational complexity also makes it less effective for real-time queries, necessitating frequent recalculations for large and dynamic networks.

HITS, in contrast, eliminates some of the vulnerabilities present in PageRank, such as susceptibility to link farming. By distinguishing between hub and authority pages, HITS ensures that relevance is derived from mutual reinforcement

within tightly connected topic-specific clusters, reducing the influence of artificial link structures. Additionally, it works well for nuanced queries, as its dual-score mechanism directly targets pages most pertinent to the query's context by finding pages called hubs that point to multiple authoritative sources concerning a general query. This semantic precision allows it to address one of the key weaknesses of PageRank. However, HITS is not without drawbacks. It is more sensitive to initial conditions and requires a carefully selected base set of pages to generate meaningful results. It is also vulnerable to topic drift, where irrelevant but highly connected pages dominate the results. Despite these limitations, HITS presents a valuable alternative for applications where contextual relevance and resistance to manipulation are paramount.

Ranking pages is, at a fundamental level, a subjective matter. Dozens of factors that cannot be computed algorithmically can sway a rank, and thus one can never obtain an algorithm that delivers a universally perfect algorithm. Nevertheless, with certain underlying assumptions and approximations, HITS and PageRank are sufficiently accurate.

6 Conclusion

The exponential growth of the internet has created an ever-increasing challenge of efficiently ranking and retrieving webpages. The two foundational algorithms that have been analyzed, *PageRank* and *HITS*, have significantly shaped this field by addressing the problem in very different ways. PageRank leverages the structure of the web as a graph, assigning importance to pages based on the quality and quantity of inbound links, while damping factors handle isolated or disconnected nodes. HITS, on the other hand, distinguishes between hubs and authorities to capture contextual relevance, offering a dynamic interplay between link types. Linear algebra plays a pivotal role in both algorithms, with matrices and eigenvalues serving as the foundation for iterative computations. Coupled with modern tools like Julia, we can model and visualize the implementation of these algorithms and thus simulate complex web structures effectively. Despite their individual limitations—PageRank’s vulnerability to link farming and HITS’s sensitivity to initialization—these approaches represent significant strides in search engine development.

As we continue to refine ranking techniques, building upon these algorithms ensures that future systems move closer to accurately reflecting the relevance and quality of information on the internet, even if the “perfect” ranking system remains elusive.

7 Bibliography

1. University System of Georgia. (n.d.). A Brief History of the Internet. Usg.edu; University System of Georgia.
https://www.usg.edu/galileo/skills/unit07/internet07_02.phtml
2. List of websites founded before 1995. (2021, May 13). Wikipedia.
https://en.wikipedia.org/wiki/List_of_websites_founded_before_1995
3. Bisht, J. (2017, August 30). Page Rank Algorithm and Implementation - GeeksforGeeks. GeeksforGeeks.
<https://www.geeksforgeeks.org/page-rank-algorithm-implementation/>
4. Amine, A. (2020, December 20). PageRank algorithm, fully explained. Medium.
<https://towardsdatascience.com/pagerank-algorithm-fully-explained-dc794184b4af>
5. How Google's PageRank Algorithm Works. (n.d.). Wwww.youtube.com.
<https://www.youtube.com/watch?v=meonLcN7LD4>
6. PageRank Algorithm - Example. (n.d.). Wwww.youtube.com.
https://www.youtube.com/watch?v=P8Kt6Abq_rM
7. Tanase, R., & Radu, R. (n.d.). PageRank Algorithm - The Mathematics of Google Search. Pi.math.cornell.edu.
<https://pi.math.cornell.edu/mec/Winter2009/RalucaRemus/Lecture3/lecture3.html>
8. Machado, J. (n.d.). LINEAR ALGEBRA APPLICATION: GOOGLE PAGERANK ALGORITHM.
<https://mathstats.uncg.edu/sites/yasaki/publications/machado-google-pagerank-linear-algebra-project.pdf>
9. Wikipedia Contributors. (2019, April 20). Random walk. Wikipedia; Wikimedia Foundation.
https://en.wikipedia.org/wiki/Random_walk
10. Wikipedia Contributors. (2019, June 18). PageRank. Wikipedia; Wikimedia Foundation.
<https://en.wikipedia.org/wiki/PageRank>
11. Rousseau, C. (2010). How Google works.
https://wiki.math.ntnu.no/_media/tma4145/2015h/google-pagerank.pdf
12. How Google's PageRank Algorithm Works. (n.d.). Wwww.youtube.com.
<https://www.youtube.com/watch?v=meonLcN7LD4>
13. HITS Algorithm - Hubs and Authorities on the Internet. (n.d.). Pi.math.cornell.edu.
<https://pi.math.cornell.edu/mec/Winter2009/RalucaRemus/Lecture4/lecture4.html>

14. Tanase, R., & Radu, R. (n.d.). PageRank Algorithm - The Mathematics of Google Search. Pi.math.cornell.edu.
<https://pi.math.cornell.edu/~mec/Winter2009/RalucaRemus/Lecture3/lecture3.html>
15. Wah, K., & Linn, W. (n.d.). Analysis of HITS Algorithm for Information Retrieval. Retrieved December 6, 2024, from
<https://onlineresource.ucsy.edu.mm/bitstream/handle/123456789/841/11086.pdf?sequence=1&isAllowed>
16. Saito, N. (2017). MAT 167: Applied Linear Algebra Lecture 24: Searching by Link Structure I.
<https://www.math.ucdavis.edu/~saito/courses/167.s17/Lecture24.pdf>
17. CS Academy. (n.d.). Csacademy.com.
https://csacademy.com/app/graph_editor/